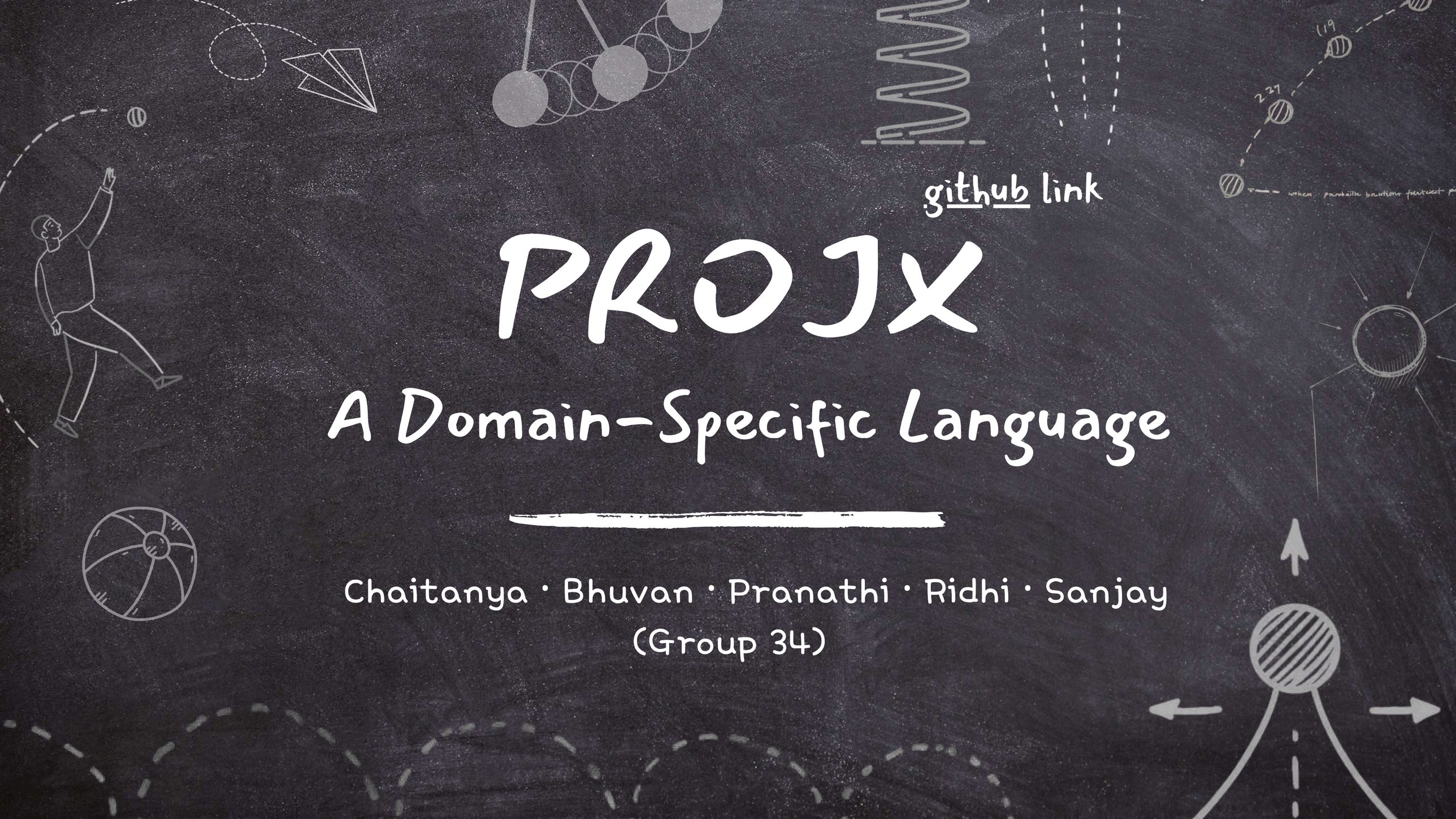
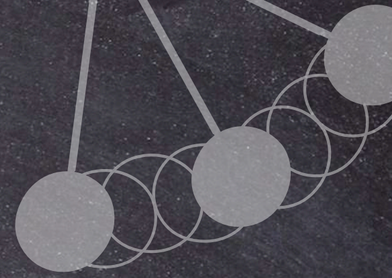


github link

PROJX

A Domain-Specific Language

Chaitanya · Bhuvan · Pranathi · Ridhi · Sanjay
(Group 34)



How ProjX Works?

PROJX processes a simulation script through a structured compilation pipeline:

Lexical Analysis

Converts input script into tokens

Parsing

Builds a structured representation (syntax tree)

Semantic Analysis

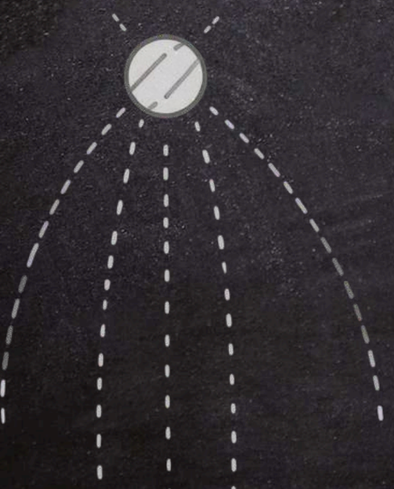
Validates logic and ensures correctness

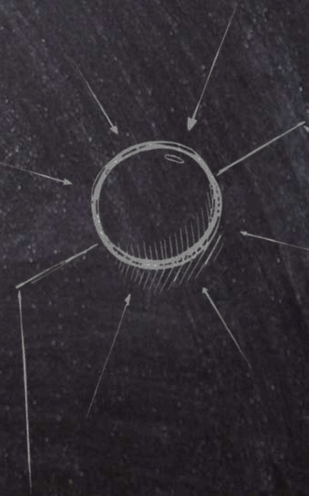
Evaluation

Computes simulation results

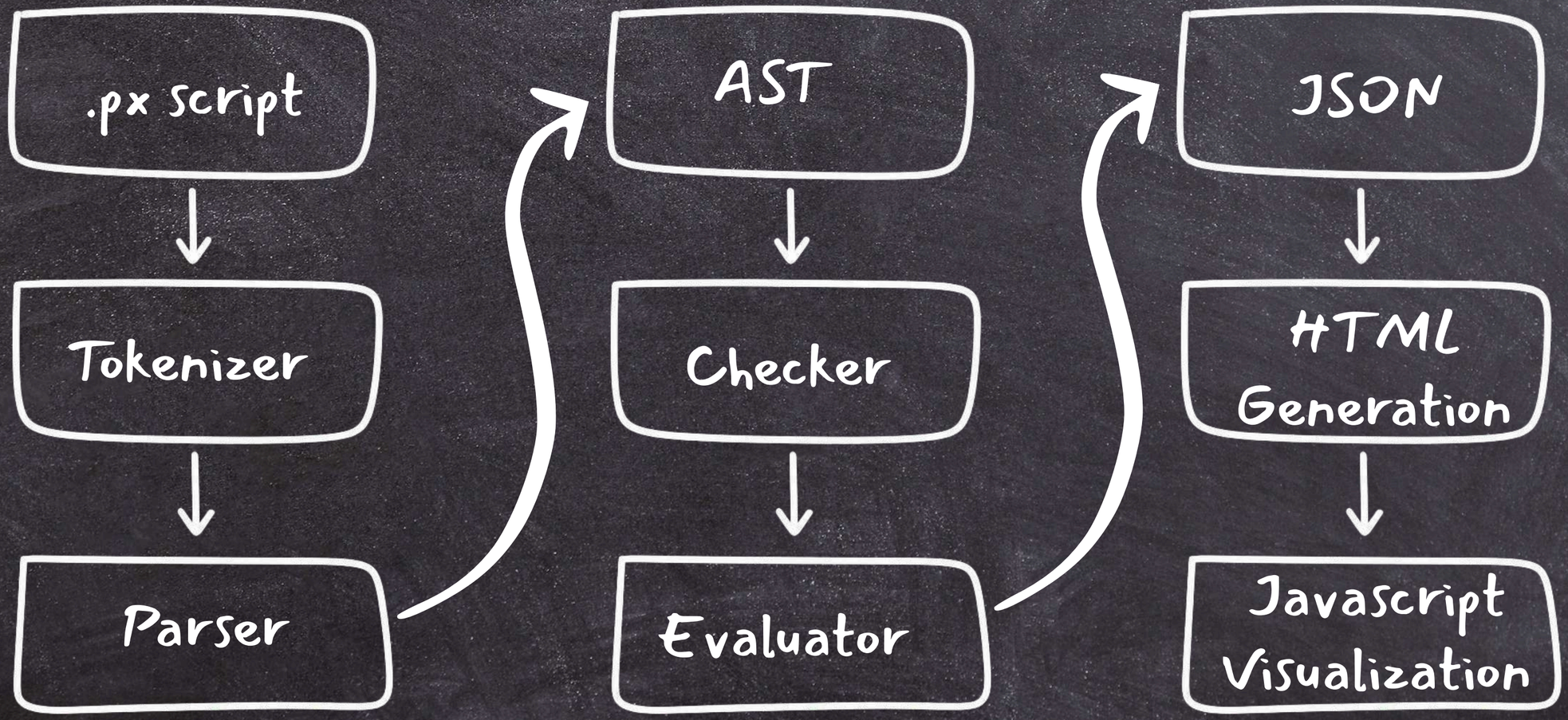
Code Generation

Produces JSON output and HTML visualizations





Execution Flow:



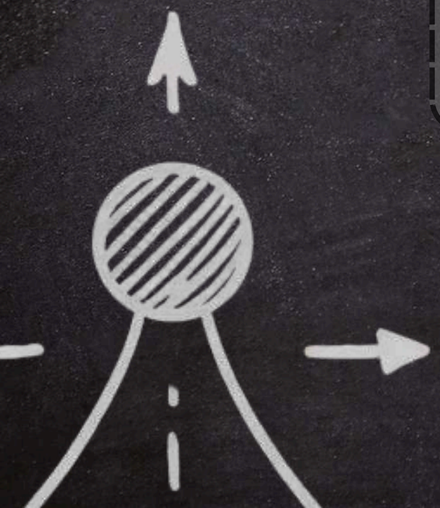
Executive Demo

Tokenization

Tool: hand-written lexer in lib/tokenizer.ml (no external lexer generator)
Recognizes keywords, identifiers, numeric literals, operators, punctuation, and # comments

Static Analysis

Tool: lib/checker.ml
Enforces declaration-before-use, mandatory angle/speed, required gravity/plot in every simulate block, valid planet names, and no duplicate let declarations



Final Output Language

JSON (emitted to stdout via lib/json_emit.ml using the yojson library)
A standalone HTML + JavaScript file — no server needed, opens directly in a browser
The frontend uses Three.js for 3D visualization and vanilla JS for 2D rendering and game mode

Evaluation & Physics

lib/eval.ml walks the AST and calls lib/physics.ml for trajectory, range, height, collision, bounce, and drag calculations

Parsing

A hand-written recursive-descent parser in lib/parser.ml builds an AST from lib/ast.ml. It manages projectile blocks, simulate blocks, fork/branch blocks, game blocks, control flow, and dot-query expressions.

LIBRARIES & TOOLS USED

1 Core Language & Build System

OCaml (v5.x) → Core implementation language
Dune → Build system & project management

2 Frontend / Output

HTML Generator → Interactive visualization
Browser-based rendering (no server required

3 Compiler & Processing Modules

tokenizer.ml → Lexical analysis
parser.ml → Syntax parsing (AST generation)
checker.ml → Semantic validation
eval.ml → Physics evaluation engine

4 Development Environment

OPAM → Package manager
CLI-based execution (dune exec projx)

5 Data Handling

Yojson → JSON generation for simulation output

6 Testing Framework

OUnit2 → Unit testing for:
Tokenizer correctness
Parser validation
Syntax & logic testing

Keywords

STRUCTURE KEYWORDS

projectile → Defines a projectile object
simulate → Starts simulation block
fork → Creates comparison branches
game → Defines interactive simulation

CONTROL FLOW

for, while, repeat → Loops
if, else → Conditions
let → Variable declaration
set → Variable update

PROJECTILE PROPERTIES

angle, speed → Motion parameters
launch_from → Initial position
mass → Object mass
drag_coefficient, cross_section → Air effects

ANALYSIS / QUERIES

range() → Distance traveled
max_height() → Peak height
collide() → Collision detection
min_dist() → Minimum distance

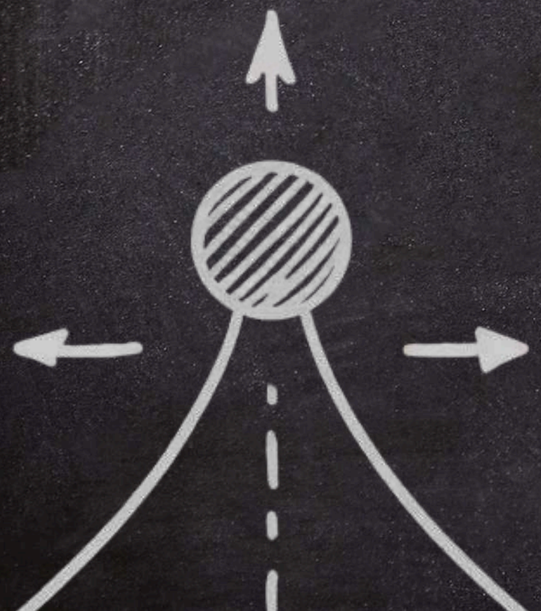
SIMULATION CONFIG

gravity → Set environment gravity
air_resistance → Enable/disable drag
wind_x, wind_y, wind_z → Wind forces
plot → Visualize trajectory

ADVANCED FEATURES

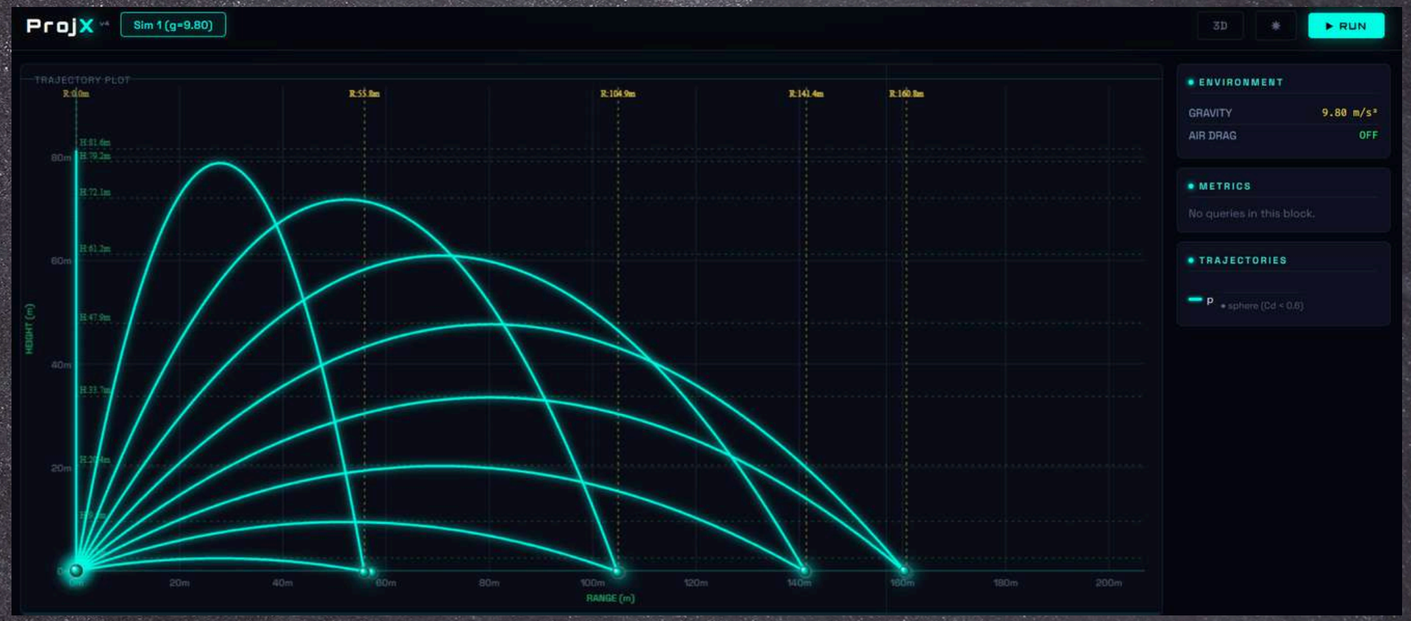
branch → Scenario variation
bounce → Rebound simulation
check → Condition validation
Dot queries → range.ball()

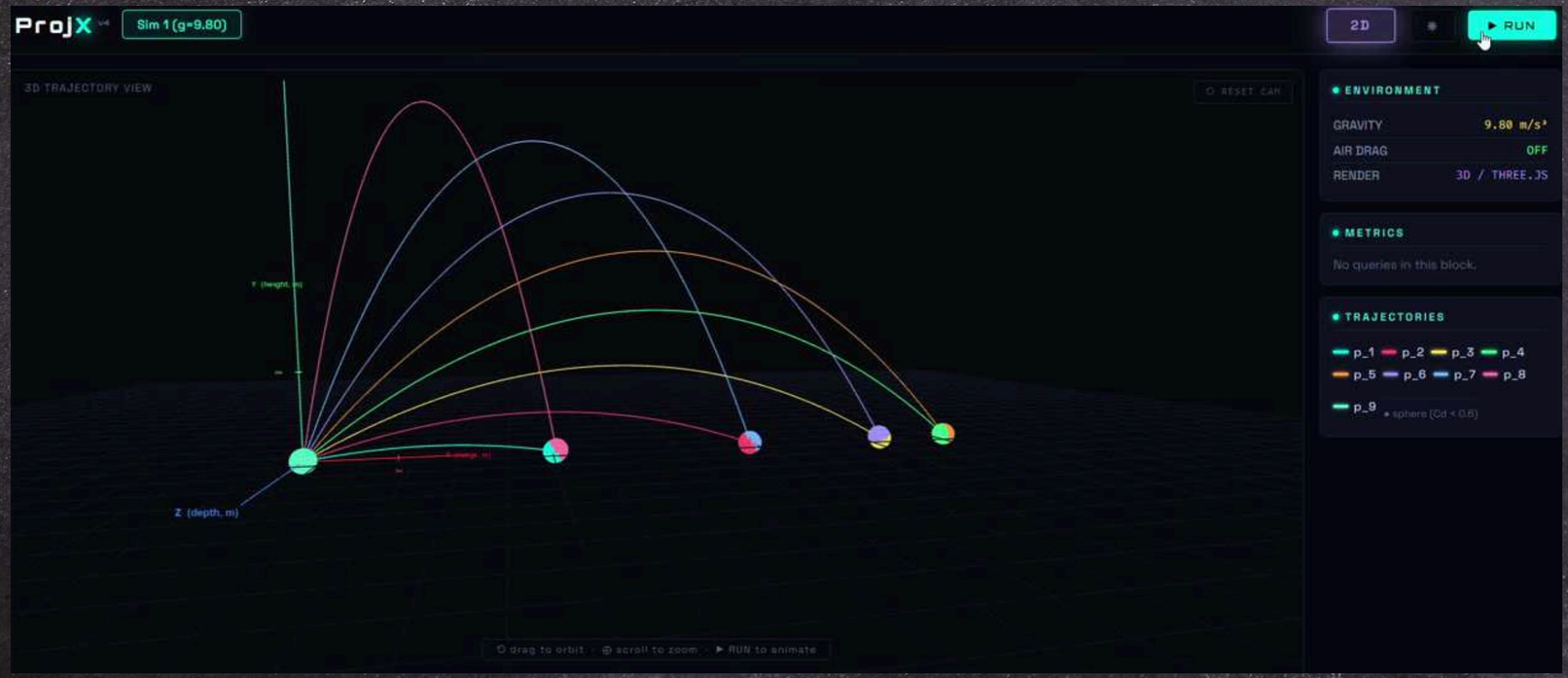
Example Program

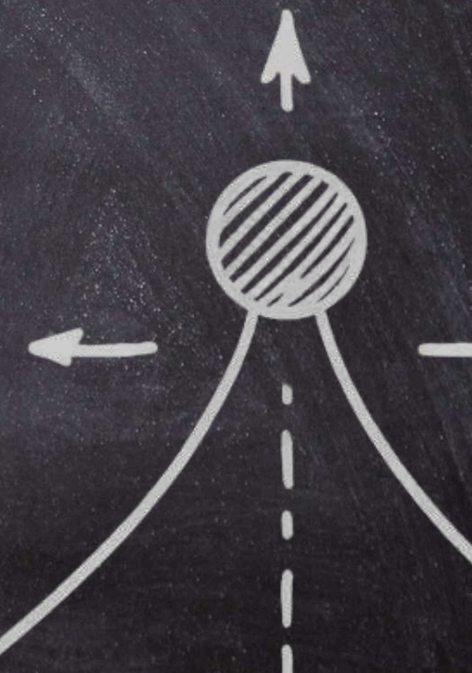


```
let ang = 10
simulate {
    gravity 9.8
    for ang from 10 to 90 step 10 {
        projectile p{
            angle ang
            speed 40
        }
        plot p
    }
}
```

Generated Output







Game Mode

Game Mode extends PROJX from static simulation to dynamic, event-driven execution.
It enables simulations to evolve over time by responding to events such as updates and collisions, allowing the modeling of interactive and realistic behaviors.
PROJX also supports environment-specific modes, including:

- Earth Mode — realistic motion under standard gravity with interactions like collisions
- Moon Mode — reduced gravity leading to different motion characteristics
- Custom Modes — user-defined physical parameters

This allows users to observe how the same simulation behaves under different physical conditions, without changing the core logic.

Different Modes

Example Program

```
game {
  planet mars
  level 4
  lives 5
}
```

Fork Block

- The fork block enables parallel or branched execution of simulation logic.
- Allows multiple simulation paths to be executed independently
- Useful for modeling simultaneous behaviors or alternative scenarios
- Each branch can define separate actions or outcomes
- Enhances expressiveness for complex and dynamic simulations

Code Example

```
projectile probe {
  angle 45
  speed 70
  launch_from (0, 0, 0)
}

fork probe {
  branch "Earth (g=9.8)" {
    gravity 9.8
    plot probe
    max_height probe
    range probe
  }
  branch "Moon (g=1.62)" {
    gravity 1.62
    plot probe
    max_height probe
    range probe
  }
  branch "Mars (g=3.72)" {
    gravity 3.72
    plot probe
    max_height probe
    range probe
  }
  branch "Jupiter (g=24.79)" {
    gravity 24.79
    plot probe
    max_height probe
    range probe
  }
  branch "Sun (g=274)" {
    gravity 274
    plot probe
    max_height probe
    range probe
  }
}
```

User Survey

"This survey serves as a structured evaluation framework to rigorously assess ProjX's usability, learnability, and practical effectiveness."

Performance & Learning

85.7% said ProjX improved understanding
71.4% results matched expectations
100% users satisfied (0 negative feedback)

Adoption & Usability

71% users had no prior experience → still found ProjX easier
100% successfully ran simulations
85.7% rated syntax easy (4–5/5)

User Acceptance

100% positive satisfaction
85.7% likely to recommend
0% dissatisfaction / failure cases

Advanced Usage

85.7% created complex simulations
Manageability rated high (majority ≥ 8/10)

Conclusion:

ProjX demonstrates high usability, strong learning impact, and excellent user acceptance.

Scan here for user form

Feedback & Improvements

Key Insights

- Low learning barrier→ Users got simulations running with minimal effort
- Highly intuitive syntax→ Rated maximum ease by all participants
- Primary challenge→ Writing queries and debugging

User Experience

All users could handle complex simulations effectively
PROJX significantly improved understanding of projectile motion
Results were mostly aligned with expectations

Improvements

- Added wind feature
- Added 3d visualisation
- Initially there was only darkmode in the visualisation it got improved now.

Summary

PROJX is a domain-specific language designed to provide a unified and structured approach to physics-based simulation and modeling. It enables users to express complex projectile behaviors using clear, high-level constructs.

The system seamlessly handles the complete pipeline from parsing and evaluation to visualization, reducing implementation complexity while maintaining flexibility.

With support for interactive execution and environment-based modeling, along with strong user validation, PROJX demonstrates both practical usability and technical depth.